SC6103 分布式航班信息系统

演示操作手册（简明版）

三台电脑 · 谁做什么 · 应该看到什么

代码版本	分支 `revised-v2`，提交 `2dcabdc`，目录 `revised_v2/`
演示日期	10 月 17 日，约 13 分钟
截图说明	来自 2026-10-11 的实际运行。截图中的 IP、端口和请求编号以现场为准

目录

1. 分工与流程
2. 演示前准备
3. 演示步骤 1–7
4. 出问题怎么办
5. 附录：讲解要点与常见问题、命令速查、纯终端备用流程

1. 分工与流程

电脑	谁	运行什么	主要操作
电脑 1	**A**	服务器（终端）	启动服务器，换实验配置，指着服务器日志讲
电脑 2	**B**	C++ 客户端（终端），监控方	监控航班、接收回调，做语义实验
电脑 3	**C**	网页客户端，订票的客户	查航班、错误输入、订座、退座

步骤	内容	投屏	时长
1	开场：三台电脑的角色	电脑 1	1.5 分
2	查航班	电脑 3	1.5 分
3	错误输入	电脑 3	1 分
4	监控回调	**电脑 2**	2 分
5	监控到期	电脑 2	1 分
6	消息编组	电脑 3	1.5 分
7	调用语义实验	电脑 2	4 分

服务器和监控方都是终端，网页只当订票的客户用。没投屏的电脑把屏幕转向老师。A 的服务器窗口在每一步都可以拿来对照。

2. 演示前准备

2.1 每台电脑（提前一天做好）

1. 下载代码（直接下载 `revised-v2` 分支）：
   `git clone --branch revised-v2 https://github.com/russellpn520-gif/6103_-_-.git`
   下载完成后，要用的文件都在 `6103_-_-` 文件夹里的 `revised_v2` 子文件夹。已经下载过的电脑，在 `6103_-_-` 里运行 `git pull` 更新。
2. **Windows**：把组群里发的 `server.exe`、`client.exe` 放进 `revised_v2` 文件夹。**Mac**：在 `revised_v2` 里运行一次 `make`。
3. 电脑 3 的网页客户端需要 Python 3.8 以上，不用装任何包。

2.2 现场

谁	操作	应该看到
全体	三台电脑连同一个手机热点	
A	查本机 IP（Windows：`ipconfig`；Mac：`ipconfig getifaddr en0`），告诉 B 和 C	例如 `192.168.43.10`
A	双击 `revised_v2` 里的启动菜单，输入 `1`	"Server listening on UDP port 8080"，10 个航班
B	双击启动菜单，输入 `6`，再输入 A 的 IP	第一行 "local UDP port 数字"，下面是 C++ 客户端菜单
C	双击启动菜单，输入 `5`，再输入 A 的 IP	浏览器自动打开，右上角绿色 **Connected** 和 `UDP :端口`
全体	防火墙弹窗勾选"专用网络"并允许；终端字体调大；C 的浏览器缩放调到 110–125%，只开一个网页标签	

启动菜单文件	数字
Windows：`demo.bat` Mac：`demo.command` （第一次被拦截时：右键 → 打开）	`1` 服务器（主流程）　`2` 服务器（语义第一轮）　`3` 服务器（语义第二轮）　`4` 服务器（请求丢失） `5` 网页客户端　`6` C++ 客户端 at-most-once　`7` C++ 客户端 at-least-once `8` C++ 客户端 请求丢失

> **回到菜单：** **Windows**：按 `Ctrl+C` → 出现 "Terminate batch job (Y/N)?" 时输入 `N` → 出现 "Press any key" 时按任意键 → 看到菜单后再输入数字。
> **Mac**：按 `Ctrl+C` → 按回车 → 看到菜单后再输入数字。
> C++ 客户端用 `0` 退出后，同样先按任意键（Mac 按回车）再回到菜单。下文把这一串简写为 "**回到菜单**"。
> 服务器日志会一直留在窗口里，并同时保存在 `revised_v2/demo_server.log`。

3. 演示步骤

步骤 1　开场

投屏　电脑 1

谁	操作	应该看到
A	展示服务器窗口，介绍三台电脑：电脑 1 服务器，电脑 2 监控方（终端），电脑 3 订票客户（网页）	服务器监听 UDP 8080，加载 10 个航班
B	指终端第一行	"local UDP port 数字"：这就是电脑 2 的客户端端口
C	指网页右上角	Connected 和电脑 3 的 UDP 端口

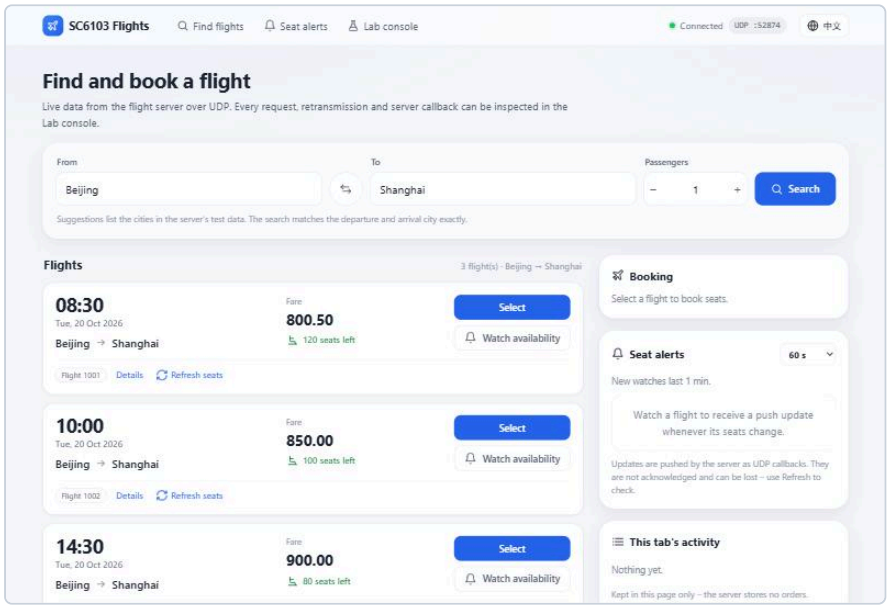
```
电脑1 - 服务器终端：./server --port=8080 --semantics=at-most-once --loss-rate=0
[02:40:13] [INFO] Starting flight server...
[02:40:13] [INFO] Invocation semantics: at-most-once (duplicate requests are answered from the reply history, never re-executed)
[02:40:13] [INFO] Simulated reply loss rate: 0.00 (seed 1791657613)
[02:40:13] [INFO] Log file: D:/ntu/SC6103-DISTRIBUTED SYSTEMS/demo_manual/hybrid_server.log
[02:40:13] [INFO] Server listening on UDP port 8080
[02:40:13] [INFO] Loading test flights:
[02:40:13] [INFO]   Flight 1001: Beijing -> Shanghai, departs 2026-10-20 08:30, airfare 800.50, 120 seats
[02:40:13] [INFO]   Flight 1002: Beijing -> Shanghai, departs 2026-10-20 10:00, airfare 850.00, 100 seats
[02:40:13] [INFO]   Flight 1003: Beijing -> Shanghai, departs 2026-10-20 14:30, airfare 900.00, 80 seats
[02:40:13] [INFO]   Flight 2001: Shanghai -> Beijing, departs 2026-10-21 09:00, airfare 820.00, 110 seats
[02:40:13] [INFO]   Flight 2002: Shanghai -> Beijing, departs 2026-10-21 15:00, airfare 870.50, 90 seats
[02:40:13] [INFO]   Flight 3001: Guangzhou -> Chengdu, departs 2026-10-22 07:30, airfare 1200.00, 150 seats
[02:40:13] [INFO]   Flight 3002: Guangzhou -> Chengdu, departs 2026-10-22 13:00, airfare 1250.00, 130 seats
[02:40:13] [INFO]   Flight 4001: Shenzhen -> Xi'an, departs 2026-10-23 11:00, airfare 1100.00, 100 seats
[02:40:13] [INFO]   Flight 5001: Hangzhou -> Wuhan, departs 2026-10-24 16:00, airfare 700.00, 140 seats
[02:40:13] [INFO]   Flight 6001: Nanjing -> Kunming, departs 2026-10-25 12:30, airfare 1500.00, 120 seats
[02:40:13] [INFO] Initialized 10 test flights
```

服务器启动

步骤 2　查航班

投屏　电脑 3

谁	操作	应该看到
C	From 填 `Beijing`，To 填 `Shanghai`，点 **Search**	3 张航班卡片：1001、1002、1003，余票 120 / 100 / 80
A	指服务器日志	来自电脑 3 的 4 个请求：1 次查航线 + 3 次查详情



电脑 3：搜索结果

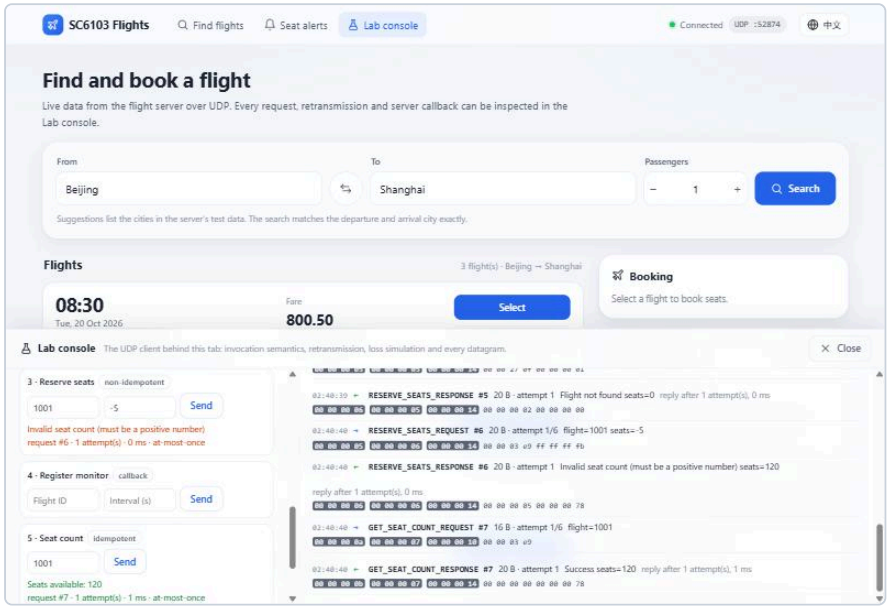
```
电脑1 - 服务器终端：电脑3 网页搜索一次 = 4 个逻辑请求（端口 52874）
[02:40:29] [INFO] Request #1 from 127.0.0.1:52874: QUERY_FLIGHT source="Beijing" destination="Shanghai"
[02:40:29] [INFO] Reply   #1 to   127.0.0.1:52874: Success, flights [1001, 1002, 1003]
[02:40:29] [INFO] Request #2 from 127.0.0.1:52874: GET_FLIGHT_INFO flight=1001
[02:40:29] [INFO] Reply   #2 to   127.0.0.1:52874: Success, Beijing -> Shanghai, departs 2026-10-20 08:30, airfare 800.50, seats 120
[02:40:29] [INFO] Request #3 from 127.0.0.1:52874: GET_FLIGHT_INFO flight=1002
[02:40:29] [INFO] Reply   #3 to   127.0.0.1:52874: Success, Beijing -> Shanghai, departs 2026-10-20 10:00, airfare 850.00, seats 100
[02:40:29] [INFO] Request #4 from 127.0.0.1:52874: GET_FLIGHT_INFO flight=1003
[02:40:29] [INFO] Reply   #4 to   127.0.0.1:52874: Success, Beijing -> Shanghai, departs 2026-10-20 14:30, airfare 900.00, seats 80
```

服务器日志：请求 #1 查航线，#2–#4 查详情

步骤 3　错误输入

投屏　电脑 3

谁	操作	应该看到
C	点顶部 **Lab console**	页面下方展开实验控制台
C	**3 • Reserve seats** 填 `9999`、`1`，点 Send	红字 Flight not found
C	**3 • Reserve seats** 填 `1001`、`-5`，点 Send	红字 Invalid seat count
C	**5 • Seat count** 填 `1001`，点 Send	Seats available: 120（库存没变）
A	指服务器日志	两条错误回复，查到的座位仍是 120
C	点 **Close** 收起控制台	



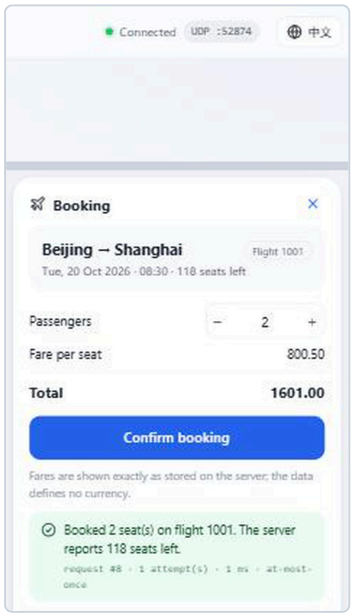
电脑 3：实验控制台的原始表单和线路日志

步骤 4　监控回调

投屏　电脑 2（终端）

谁	操作	应该看到
C	**先准备：**在 1001 卡片点 **Select**，人数调到 2（这一步不发请求）	右侧打开预订面板
B	在终端输入 `6` → `1001` → `30`	"Monitor registered … Waiting for updates"，终端暂时不能输入
C	看到 B 那边出现 "Waiting for updates" 后，**马上**点 **Confirm booking**	"Booked 2 seat(s)… 118 seats left"
B	不操作	终端立刻出现绿色 **[Callback] Flight 1001 seat availability is now 118**
C	紧接着在预订面板下方 **Return seats** 填 `2`，点 **Return**	"Returned 2 seat(s)… 120"
B	不操作	再出现一行绿色 **[Callback] … is now 120**
A	指服务器日志	订座和退座后面各有一行 `Callback ->`，端口 = 电脑 2 终端第一行的 "local UDP port"

> **监控时长 30 秒，从 B 输入 `30` 按回车那一刻开始算**，订票不会让它延长。所以顺序是：C 先选好航班和人数 → B 开始监控 → C 马上确认、再退座（两下约 10 秒）。剩下的时间用来讲回调原理，到期后做步骤 5。

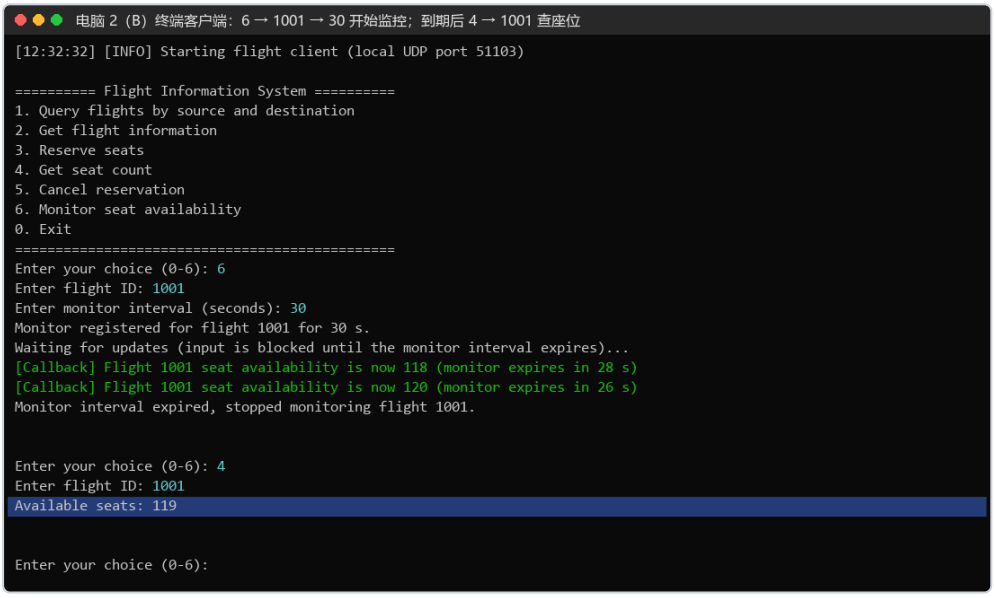


电脑 3（网页）：订座成功，服务器返回余票 118

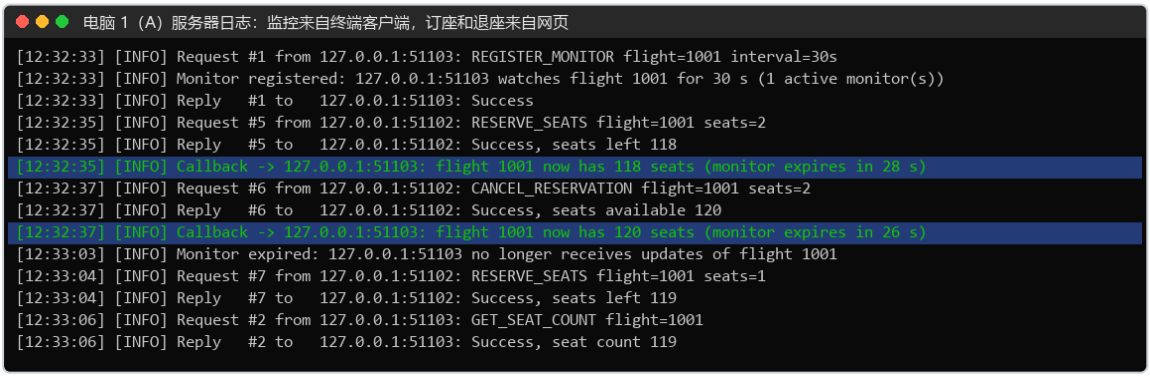
步骤 5　监控到期

投屏　电脑 2（终端）

谁	操作	应该看到
全体	等到 30 秒结束（约再等 20 秒）	电脑 2 显示 "Monitor interval expired"，菜单重新出现；服务器打印 "Monitor expired"
C	人数改为 1，点 **Confirm booking**	订座成功，余票 119
B	不操作	没有新的 [Callback]
A	指服务器日志	这次订座后面没有 `Callback ->`
B	输入 `4` → `1001` 主动查座位	Available seats: 119



电脑 2（终端）：监控 30 秒，收到两次回调（118、120），到期后查到 119



电脑 1（服务器）：订座、退座后各有一行 Callback，监控到期后的订座没有 Callback

步骤 6　消息编组

投屏　电脑 3

谁	操作	应该看到
C	点 **Lab console**，在右侧线路日志里找到 `RESERVE_SEATS_REQUEST`（步骤 4 那次订座）	一行 20 字节的十六进制
A	对照下表逐段讲字节；再在服务器窗口里找到编号相同的那条请求（服务器还没重启，日志就在窗口里）	请求编号、来源端口和字段都对得上
C	讲完点 **Close**	



电脑 3 的线路日志：订座请求和回复

字节	十六进制	字段	值
0–3	`00 00 00 05`	消息类型	5 = 订座请求
4–7	`00 00 00 08`	请求编号	8（现场编号可能不同）
8–11	`00 00 00 14`	总长度	20 字节
12–15	`00 00 03 e9`	航班号	1001
16–19	`00 00 00 02`	座位数	2

步骤 7　调用语义实验

投屏　电脑 2（终端）

> **开始前：**C 从现在起不要再动网页，否则会打乱服务器的丢包顺序。网页不用关。

第一轮：at-least-once

谁	操作	应该看到
A	服务器窗口**回到菜单**，输入 `2`	开头几行是 "at-least-once"、"reply loss rate: 0.50"。**出现 "Failed to bind" 说明旧服务器还占着端口**，先把它关掉
B	终端输入 `0` 退出客户端，**回到菜单**，输入 `7`，IP 直接回车	新的 C++ 客户端（at-least-once）
B	订座 3 次：每次输入 `3` → `1001` → `1`	有几次会等几秒、出现重传提示，这是正常的
B	查座位：`4` → `1001`	**Available seats: 113**（订 3 座却少了 7 座）
A	**重启前**指着服务器窗口讲完第一轮	4 次 "executing AGAIN"，每次多扣一座，最后剩 113

第二轮：at-most-once（第一轮的日志不会清掉，往上滚还能看到，也保存在 `revised_v2/demo_server.log`）

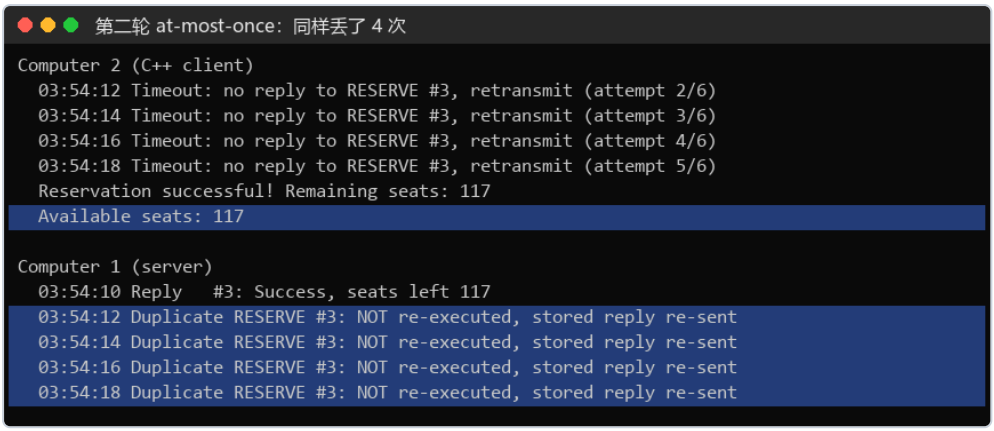
谁	操作	应该看到
A	**回到菜单**，输入 `3`	开头几行是 "at-most-once"、"reply loss rate: 0.50"，没有 "Failed to bind"
B	终端输入 `0` 退出，**回到菜单**，输入 `6`，IP 回车	新的 C++ 客户端
B	和第一轮完全一样：订座 3 次，再查座位	**Available seats: 117**（正好少 3 座）
A	指服务器日志	重复请求标着 "NOT re-executed"

第一轮 at-least-once：第 3 次订座的回复丢了 4 次

```
Computer 2 (C++ client)
  03:53:55 Timeout: no reply to RESERVE #3, retransmit (attempt 2/6)
  03:53:57 Timeout: no reply to RESERVE #3, retransmit (attempt 3/6)
  03:53:59 Timeout: no reply to RESERVE #3, retransmit (attempt 4/6)
  03:54:01 Timeout: no reply to RESERVE #3, retransmit (attempt 5/6)
  Reservation successful! Remaining seats: 113
  Available seats: 113

Computer 1 (server)
  03:53:53 Reply   #3: Success, seats left 117
  03:53:55 Duplicate RESERVE #3: executing AGAIN
  03:53:55 Reply   #3: Success, seats left 116
  03:53:57 Duplicate RESERVE #3: executing AGAIN
  03:53:57 Reply   #3: Success, seats left 115
  03:53:59 Duplicate RESERVE #3: executing AGAIN
  03:53:59 Reply   #3: Success, seats left 114
  03:54:01 Duplicate RESERVE #3: executing AGAIN
  03:54:01 Reply   #3: Success, seats left 113
```

第一轮：服务器把每份重传都再执行一次（executing AGAIN），最后剩 113



第二轮：重传被认出来，不再执行（NOT re-executed），最后剩 117

语义	订座次数	订座请求重传	查座位重传	服务器执行订座	最终座位
at-least-once	3	4	2	7	**113（错误）**
at-most-once	3	4	2	3	**117（正确）**

> 数字由随机种子决定。服务器每轮新启动、操作和上面完全一样时，结果就是 113 和 117；多做或少做一步数字会变，但两轮的对比结论不变。

可选：请求丢失（时间不够就跳过）

谁	操作	应该看到
A	**回到菜单**，输入 `4`	at-least-once，不丢回复
B	终端 `0` 退出，**回到菜单**，输入 `8`；订座 3 次，再查座位	**117**；服务器日志没有 "Duplicate"

收尾（一定要做）

谁	操作	应该看到
A	**回到菜单**，输入 `1`	恢复主流程服务器，座位回到初始值
B	终端 `0` 退出，**回到菜单**，输入 `6`	回到普通的 C++ 客户端
C	之后如果还要用网页，先重新点一次 Search	余票重新从服务器读取

4. 出问题怎么办

现象	处理
网页右上角不是 Connected	看桥的菜单窗口是否还开着；刷新网页
搜索或订座提示没有回复	① A 的服务器是否在运行；② B/C 输入的 IP 对不对（菜单选 5 时重新输入）；③ 是否连同一个热点；④ A 的电脑防火墙是否允许 server
订座成功，但电脑 2 收不到更新	① 关注是否已到期（显示 Watch ended）；② 服务器日志有没有 `Callback ->`、端口是不是电脑 2 的；③ 电脑 2 防火墙允许 python 在专用网络接收。不要直接关防火墙
余票和预期不同（例如预期 118 实际 115）	服务器还在用语义实验的配置：A **回到菜单**输入 `1` 重启，B/C 网页重新 Search
菜单选 5 提示找不到 Python	安装 python.org 的 Python 3，安装时勾选 "Add python.exe to PATH"
服务器提示 "Failed to bind"	8080 端口被另一个服务器占着：把其他服务器窗口都**回到菜单**，只在一个窗口里重新选
语义实验没有重传，座位数也不对（例如 111、110）	新服务器没起来，客户端连的还是旧服务器：看服务器窗口有没有 "Failed to bind"。处理好后服务器和客户端都重新开始这一轮
热点下电脑之间连不上	换一部手机开热点；实在不行用一台电脑开多个窗口演示，并向老师说明

附录 A　讲解要点与常见问题

话题	要点
怎么证明用的是 UDP	三条一起讲：桥运行在客户端电脑上、指向服务器 IP；服务器和桥的代码都创建 `SOCK_DGRAM` 套接字；网页线路日志和服务器日志的请求编号、来源端口一一对应。只看 IP 和端口不能证明是 UDP
网页是不是用了 HTTP	浏览器到本机的桥是 HTTP；桥到服务器是我们自己的 UDP 协议，桥用 Python 重新实现了编组，所以客户端和服务器是两种语言
消息格式	12 字节头（类型、请求编号、总长度，大端 uint32）+ 消息体；字符串 = 4 字节长度 + UTF-8 字节；float 按 IEEE-754 位模式发送；时间 = 5 个 int32
为什么一次搜索是 4 个请求	查航线只返回航班号，页面再为每个航班查一次详情
什么时候发回调	订座或退座成功，并且该航班有尚未过期的监控注册。回调不确认、不重传，丢了可以随时主动查座位（终端输入 4，网页点 Refresh）
at-least-once 和 at-most-once	两者客户端都超时重传，区别在服务器：at-least-once 每份都执行；at-most-once 按（IP，端口，请求编号）认出重复，返回保存的回复
请求丢失和回复丢失	请求丢了服务器没收到，重传后只执行一次；回复丢失或晚于超时到达时，服务器已执行过，at-least-once 会再执行一次
去重为什么要带 IP 和端口	每个客户端的请求编号都从 1 开始，只用编号会把两个客户端的 #1 当成重复
回复历史保存多久	60 秒，长于客户端 12 秒的重传窗口（2 秒 × 6 次）
两个自定义操作	查座位数（幂等，只读）；取消订座（非幂等，每次都退座，不能超过已订数量）
超时能说明没执行吗	不能。网页对订座和退座显示"请求可能已经执行，请勿立即重复提交"
为什么语义实验用 C++ 客户端	另一套真实客户端；实验已反复验证；网页搜索附带的详情请求会打乱丢包顺序

附录 B　命令速查（不用菜单时）

在 `revised_v2` 目录执行。Windows 用 `.\server.exe`、`.\client.exe`、`python`。

```
# 主流程（菜单 1、5、6）
./server --port=8080 --semantics=at-most-once --loss-rate=0
python3 web/bridge.py --server=<IP> --port=8080 --open
./client --server=<IP> --port=8080 --semantics=at-most-once

# 语义实验（菜单 2+7、3+6、4+8）
./server --port=8080 --semantics=at-least-once --loss-rate=0.5 --seed=6103
./client --server=<IP> --port=8080 --semantics=at-least-once
./server --port=8080 --semantics=at-most-once --loss-rate=0.5 --seed=6103
./client --server=<IP> --port=8080 --semantics=at-most-once
./server --port=8080 --semantics=at-least-once --loss-rate=0
./client --server=<IP> --port=8080 --semantics=at-least-once --loss-rate=0.5 --seed=7

# 本机兜底：自动对比实验（只在一台电脑上跑）
./tools/test_semantics.sh
```

附录 C　纯终端备用流程（电脑 3 的网页出问题时）

先重置：停止当前操作。A 的服务器**回到菜单**，输入 `1` 重新启动。C 在运行网页客户端的菜单窗口里**回到菜单**，输入 `6` 改用 C++ 客户端，IP 直接回车。B 本来就是终端，不用换。然后代替步骤 2–5 做下面这些：

谁	输入	应该看到
C	`4` → `1001`	Available seats: 120（确认是初始状态）
C	`1` → `Beijing` → `Shanghai`	Found 3 flights
C	`2` → `1001`	出发时间、票价 800.50、120 座
C	`3` → `9999` → `1`	Error: Flight not found
B	`6` → `1001` → `30`	开始监控，窗口暂时不能输入（30 秒内做完下面两步）
C	`3` → `1001` → `2`	B 立刻出现绿色 [Callback] … 118
C	`5` → `1001` → `2`	B 再收到 … 120
全体	等待到期	B 显示 Monitor interval expired
C	`3` → `1001` → `1`	订座成功，B 不再收到回调

这样六项服务都演示到了：查航线、查详情、订座、查座位、取消、监控。步骤 6 没有网页时，直接用字节表配合源码讲；步骤 7 不受影响。